Pii

Developer's Manual: Architecture & Technical Analysis

Pii Project

Version 0.1.0

pi-improved

Preface

Pii is a single static Rust binary. This manual explains *how it works*: the architecture decisions, the graph engine, the capability and safety model, the event-sourced journal, the persistence layer, and the technical analysis that shaped them.

The companion *User Manual* covers installation, configuration, operation, and debugging for system administrators. The two manuals complement each other: this one answers *how it works*, the other answers *how to operate*. Cross-references are marked inline.

Contents

Contents iii

1 Architecture Overview 1
- 1.1 The four pillars . . . 1
- 1.2 The workspace . . . 1
- 1.3 Feature gating . . . 1

2 The Graph Engine 3
- 2.1 The specification model . . . 3
- 2.2 Node kinds . . . 3
- 2.3 If/Then/Else — the `route` node . . . 4
- 2.4 Loops — bounded cycles via `loop_budget` . . . 4
- 2.5 Capabilities and Effects (safety model) . . . 4
- 2.6 Execution: tick loop, batch drain, event-sourced journal . . . 5

3 Persistence 6
- 3.1 The append-only journal . . . 6

4 The Control Plane 7

5 The Self-Improving Boss 8
- 5.1 The heal-fix loop . . . 8
- 5.2 Protected paths . . . 8

6 Technical Analysis 9
- 6.1 Convergent architecture (dsh comparison) . . . 9
- 6.2 Context management (Slice C) . . . 9
- 6.3 The graph engine vs. git log . . . 9
- 6.4 Where to go for more . . . 9

Architecture Overview 1

Pii is a single-process, event-sourced monolith: one redb store, single-writer; journal + bus + approval gate + budget are coherent in-process. That monolith is its strength. The full vision and the four pillars are in the SSOT at `_docs/01-overview/001-pii-vision.md`.

1.1 The four pillars

1. **Coding agent core** (from `pi_agent_rust`) — sub-100ms cold start, structured concurrency, capability-gated `Ctx`, JSONL sessions, bash mediation.
2. **Assistant/gateway** (from `zeroclaw`) — channels, LLM providers, tools, SOP engine, WASM plugin host, hardware HAL.
3. **Cognitive memory + model tiering** (from `yantrikclaw`) — entities/relationships, tier-aware tool selection for model sizes 0.5B→100B+.
4. **Self-improvement loop** (from `hermes-agent`) — skill extraction, procedural templates, trajectory logging.

1.2 The workspace

The workspace is a Rust workspace of crates. The kernel ABI is `pii-abi` (traits + types only, no transport deps), enforced by `xtask deps-check` and a hard file-size gate. The crates:

Crate	Role
`pii-abi`	kernel ABI: traits + types, no transport deps
`pii-core`	capability context, budget, child-process
`pii-store`	redb persistence (single-writer MVCC ACID)
`pii-graph`	the graph automation engine
`pii-cli`	the `pii` binary (CLI entry point)
`pii-proto`	UDS JSON-RPC protocol + client
`pii-model`	model providers
`pii-net`	the socket chokepoint / egress policy
`pii-memory`	global memory store + embedder
`pii-worker`	supervised sub-process pool
`pii-tools`	the toolset (fs, edit, bash, glob, grep, http, memory)
`pii-channels`	outbound messaging (Discord, . . .)
`pii-server`	the daemon: Engine, registry, runs, sessions
`pii-tui`	the terminal UI
`pii-web`	the read-only web viewer
`pii-mcp`	MCP server + client
`pii-evolve`	the self-improving boss (feature-gated)
`xtask`	build/CI helpers

1.3 Feature gating

Pii is intentionally a monolith, but the installer may want a smaller or more hardened binary. Cargo **features** are the lever for *what's in*

the build — NOT for *what's a microservice*. A feature trims the compile; it does not create a process/IPC boundary. The convention: gate surfaces with risk or optional deps; never gate the engine substrate. See `_docs/02-guides/feature-gating.md`.

Design decision

Two subsystems are a security win as opt-in features: `channels`, `tls`, and `github` all do network egress and may carry a token. A feature that *excludes the code* is stronger than a runtime config that *disables* it — the dialing code isn't compiled in, so it cannot be reached.

The Graph Engine | 2

Pii ships a deterministic, statically-validated **graph automation engine** (`crates/pii-graph`) that is the control plane under every agent behavior. A graph is a dataflow/control-flow program expressed in TOML whose execution is **event-sourced**: every step appends to an append-only run journal, and a run can be resumed by replaying that journal.

A graph is NOT a script. It is a typed assembly of **nodes** (pure functions, LLM agents, tools, approval gates, timers), connected by **edges** that carry data or ordering, with explicit **join policies** for fan-in, **route** nodes for if/then/else, **loop budgets** for bounded cycles, **capability declarations** at three layers, and **effects declarations** that decide parallel vs serial execution.

The full engine design is in the SSOT at `_docs/06-research/006-pii-graph-engine.md`. The operational commands (`pii graph lint|show|run|resume|trace`) are in the User Manual, Chapter 4.

2.1 The specification model

```
GraphSpec { name, entry: NodeId, caps, limits, nodes: [NodeSpec], edges: [Edge] }
NodeSpec  { id, kind, join: JoinPolicy, loop_budget: Option<u32>,
            retry: RetryPolicy, caps, cfg: serde_json::Value }
Edge      { from, port: PortName, to, kind: EdgeKind }
```

- **Edges**: `EdgeKind::Data` (value-carrying dependency: readiness + payload) vs `EdgeKind::Order` (ordering-only: readiness, no payload).
- **JoinPolicy**: `All` (default — every inbound edge must fire) vs `Any { cancel_losers }` (any one inbound edge fires the node; optionally cancels sibling branches once one wins).
- **Limits**: `max_steps`, `max_wall_ms`, `max_tokens`, `max_parallel` — per-run budgets enforced in the tick loop; exhaustion is terminal and non-retryable.
- **RetryPolicy**: per-node `max` retries on retryable failures.
- **Expression language**: deliberately tiny — `$.path.into.context` object-member access only; no arrays, no functions. Used by `transform.expr`, `route.on`, and tool args.

2.2 Node kinds

Kind	Layer	Role	Output
`transform`	pure built-in	JSON reshape via `expr`	ok / err
`route`	pure built-in	branch on `on`-expr via `rules[]`	one per
`emit`	built-in	write a value to a file (fs_write cap)	ok / err
`wait`	pure built-in	sleep/fire-or-timeout	fired / t
`timer`	pure built-in	delay `in_ms` or 5-field `cron`	ok
`agent`	server	LLM agent: system prompt, tools allowlist, `max_turns`, `loop_budget`, compaction	ok / err
`tool`	server	human-authored command (e.g. `bash`)	ok / err
`gate`	server	suspend the run for human approval	ok
`heal_detect`	server	self-healing ledger trigger	ok

Kind	Layer	Role	Output
`memory`	server	standalone `consolidate` maintenance op	ok
`worker`	server	supervised sub-process pool	ok
`crash`	debug	deliberate failure for tests	—

2.3 If/Then/Else — the `route` node

Branching is not a separate "cond" node: `route` maps a `$.` expression to output ports via an ordered rule list, and edges wire each port to a different downstream node. The last rule without an `eq` is the else.

```
[[node]]
id = "gate"
kind = "route"
on = "$.payload.exit_code"
rules = [
  { eq = 0, port = "pass" },
  { port = "fail" },          # fallthrough / else
]
```

2.4 Loops — bounded cycles via `loop_budget`

Cycles are legal when a member node declares `loop_budget = N`. `is_in_cycle` (tarjan SCC) detects cycles; `reset_loop` re-runs the loop body genuinely instead of reading stale state. Exceeding the budget is a terminal failure.

```
fix [agent] (entry) (loop_budget=3)
 |-- ok --> test
test [tool: bash]
 |-- ok --> gate
gate [route: $.payload.exit_code]
 |-- pass --> done
 |-- fail --> retry
retry [transform: $.payload]
 ^-- loops back to "fix" (budget=3)
done [transform: $.payload]
```

2.5 Capabilities and Effects (safety model)

Capabilities (`crates/pii-core/src/cap.rs`): bit flags — FS_READ, FS_WRITE, PROC, NET, . . . Effective node permissions are a **three-layer intersection**: `run caps ∩ graph caps ∩ node caps`. A denial at any layer fails the node with `capability denied`. This is a real bug class: in 2026-09-05 the live `ollama:` provider failed because NET was missing at the run layer AND at the graph/node layers — all three had to be fixed for a real LLM call to work. (Operational symptom: see the User Manual, Chapter 7.)

Effects (`crates/pii-abi/src/effects.rs`): each node kind declares static effect classes (PURE, READS_FS, WRITES_FS, NET, SPAWNS_PROC, MUTATES_EXTERNAL, LONG_RUNNING). The scheduler partitions each batch: `PURE | READS_FS` run in parallel (bounded by `max_parallel`), `WRITES_FS | MUTATES_EXTERNAL` run sequentially. Effects fail closed — a node that does not declare its effects is treated as fully serialising.

2.6 Execution: tick loop, batch drain, event-sourced journal

1. **Tick loop**: propagate dead branches to a fixpoint (journaling each new `NodeDead`); fail LOUDLY on an unsatisfiable join; drain ALL ready nodes into one batch; check loop budgets; dispatch.
2. **Partition**: parallel-safe nodes run in bounded thread chunks; serialising nodes run in order. Results apply as they arrive; `cancel_losers` runs after each result.
3. **Journal**: EVERY event appends to the run's append-only journal in redb — `Started`, `TickStarted`, `NodeStarted` (journaled BEFORE the node runs, so an interrupted node is visible as "may have executed"), `NodeFinished`, `NodeFailed`, `NodeCancelled`, `NodeDead`, `Suspended`, `Resumed`, `Finished`. A header (status + next-seq) is persisted with each append; the blackboard snapshots every 32 events and unconditionally on suspend.
4. **Terminal dispositions**: Running → Succeeded | Failed | Suspended (human approval) → Resumed.
5. **Resume**: reads the journal, recovers the input from the content-addressed store, replays every event to rebuild state, re-propagates dead, appends `Resumed`, and continues. A node with a terminal event is never re-invoked; a `NodeStarted` with no terminal event re-runs.

The **seq** counter is the single source of run ordering — the same number the TUI shows as the live `[seq 0x...]` indicator and that `pii graph trace` prints per event. (Operational: see the User Manual, Chapter 7.)

Persistence 3

Pii uses **redb** (single-writer MVCC ACID, pure Rust, no C dep for static musl). Consequence: the daemon owns the DB; a UDS control channel is the access path. The store is a single-file redb database.

The daemon is the single writer. The control socket is local-only Unix (mode 0600), and peer-credential authorization rejects a peer whose uid differs from the daemon's. (Operational: see the User Manual, Chapter 8.)

3.1 The append-only journal

The run journal is an append-only log of typed `RunEvents` — the single source of truth for the whole run. The LLM message history is derived by pure projection, never stored separately; replay is re-derivation. This is the same pattern DeepSeek Harness (dsh) uses, and it is the emergent standard for agent state management — two independent implementations (dsh, pidag/pi) validate it. See `_docs/06-research/005-dsh-vs-pii-harness-analysis.md`.

The Control Plane | 4

`pii-server` (`Engine`) owns the registry (built-ins + agent/tool/gate/heal/memory/worker), spawns runs (`run.start`, `run.resume`, `run.cancel`), tracks sessions, and streams events (`event.subscribe` — replay-since + live, bounded queue, `Lagged` on overflow).

The daemon socket is local-only Unix (0600). The full method surface:

Method	Purpose
`sys.hello`	handshake
`sys.status`	server status
`sys.version`	server version
`sys.shutdown`	shut the daemon down
`sys.reload`	reload config (channels/workers), never restarts
`session.create`	create a session
`session.prompt`	prompt a session
`session.interrupt`	interrupt the session's active run
`session.state`	session snapshot
`session.list`	list sessions
`session.close`	close a session
`run.start`	start a graph run
`run.get`	run header
`run.list`	list runs
`run.cancel`	cancel a run
`graph.apply`	store/replace a graph
`graph.get`	fetch a graph
`graph.list`	list graphs
`graph.lint`	lint a graph source
`event.subscribe`	subscribe to events
`event.unsubscribe`	unsubscribe
`worker.list`	list workers and their state
`worker.reload`	reload a worker (drain → respawn)
`approve.list`	list pending approvals
`approve.resolve`	resolve an approval (allow/deny)
`heal.list`	list heal-attempt ledger entries
`heal.get`	fetch a heal entry
`cron.list`	list cron entries
`channel.list`	list configured channels
`memory.search`	memory search
`memory.put`	memory put
`memory.consolidate`	memory consolidate

The thin CLI subcommands that call these methods are in the User Manual, Chapter 4.

The Self-Improving Boss | 5

The self-improving boss (`pii-evolve`) is a feature-gated crate that runs the boss loop as a real pii graph on the daemon — dogfooding (008 §3/§4). Every 30 minutes it runs the silent-noop-check gate on the daemon itself; a detected failure escalates through a `gate` node so a human approves before anything heals. No FS_WRITE anywhere in the watch graph; no agent/model node — detection, forwarding, and the approval decision are all deterministic nodes.

The deep design is in the SSOT at `_docs/06-research/008-pii-evolve.md`. The opus review and the remaining-work plan are at `009-` and `012-`.

5.1 The heal-fix loop

The heal-fix graph is a complete localize→patch→ validate→escalate loop: `probe` (bash, runs the test command in a worktree) → `route_probe` on exit code → `fix` (agent with fs tools, no bash) → `retest` → `route_retest` → `summarize` (git diff) → `escalate` (gate) → `done`, with a `retry` back-edge and `loop_budget = 2`.

5.2 Protected paths

The boss cannot patch its own oracle. `PathScope` gains a deny-wins builder; `run_caps` adds a protected-path deny list for scoped runs; `heal-collect` refuses test-deletion/protected-path patches. This is the ordering guardrail that prevents a model from deleting the test that proves it worked.

Technical Analysis 6

6.1 Convergent architecture (dsh comparison)

DeepSeek Harness (dsh) validates pi-improved's core architecture decisions — event-sourced journal, derived state, capability attenuation, append-only durability — done in TypeScript-with-plugins; pi-improved is the same pattern in a static Rust binary with a real graph engine on top. The transferable upgrades are the engineering details dsh has refined further (write-behind batching, reader-side interrupted-turn repair, per-tool timeouts, single-writer leases) — not its plugin metaphysics. See `_docs/06-research/005-dsh-vs-pii-harness-analysis.md`.

6.2 Context management (Slice C)

Context management is the one P4.7 slice that is unbuilt. The reference design to steal is `dsh-context` (a DeepSeek Harness plugin): context insight should be a **projection over the existing journal**, reading the harness's own token accounting, with a delta-mode diagnostic that directly targets the `compaction.rs:69` no-op. See `_docs/06-research/013-dsh-context-slice-c-reference.md`.

6.3 The graph engine vs. git log

The lane-allocation algorithm from `git log -graph` was ported to render pii graphs as ASCII. The one thing that does not port: loops. Git's DAG has no cycles; pii graphs genuinely cycle (`loop_budget`). Design decision: lay out nodes in topological order and render each loop closure as a separate annotation line, never as a lane. See `_docs/06-research/006-pii-graph-engine.md`.

6.4 Where to go for more

- **Operation**: the companion User Manual.
- **Research SSOT**: `_docs/06-research/` (numbered research write-ups).
- **Guides**: `_docs/02-guides/`.
- **Vision**: `_docs/01-overview/001-pii-vision.md`.